AI Agent 要件定義書

1. プロダクト概要

1.1 AI Copilot とは

Slack ライクなチャットアプリのサイドバーに組み込まれた AI アシスタント。
ユーザーがチャットの会話を参照しながら AI に質問できる。
チャットの文脈を理解した上で回答するため、「今の話題について教えて」のような自然な質問が可能。

1.2 ハッカソンにおける位置づけ

AI Copilot はこのプロダクトの核心機能。チャット基盤は AI を動かすための土台であり、AI が会話をサポートする体験がプロダクトの価値の源泉。

1.3 技術スタック

項目	技術
フレームワーク	Mastra + Hono
言語	TypeScript (Node.js 20+)
LLM（ローカル開発）	Gemini API（gemini-2.5-flash）
LLM（本番）	Vertex AI（gemini-2.5-flash、Workload Identity 認証）
話題検出	Gemini 2.5 Flash（軽量呼出）
スレッド・会話管理	Mastra Memory（@mastra/memory + @mastra/pg）
コンテキストキャッシュ	Redis（ioredis で Go 側が書き込んだ Sorted Set を読取）
バリデーション	Zod
JWT 検証	jose

2. 機能要件

2.1 AI スレッド管理

ユーザーは AI との会話を「スレッド」単位で管理する。各スレッドはスコープ（参照範囲）を持つ。

機能	説明	フロント要件
スレッド作成	タイトル + スコープを指定して新規作成	8-3
スレッド一覧	ユーザーの全スレッドを updated_at 降順で返す	8-2
タイトル編集	スレッドのタイトルを変更	8-4
スレッド削除	スレッドと配下の全メッセージを削除	8-5

2.2 AI 質問（メインフロー）

ユーザーが AI に質問すると、チャットの会話コンテキストを踏まえてストリーミングで回答する。

機能	説明	フロント要件
メッセージ送信	ユーザーが AI にテキストを送信	8-6
ストリーミング応答	SSE でトークン単位の逐次表示	8-7
コンテキスト参照	スコープに応じたチャット会話を AI に渡す	8-10〜8-13
話題検出	チャンネルスコープ時、直近の話題だけに絞り込む	—
生成中止	ストリーミング途中でクライアントが切断	8-14

2.3 会話履歴

機能	説明	フロント要件
履歴取得	スレッド内の過去の user/assistant メッセージ一覧	8-8

2.4 スコープ（コンテキスト参照範囲）

AI スレッドは 3 つのスコープを持つ。スコープによって AI に渡すコンテキストが変わる。

スコープ	channel_id	thread_root_id	AI の参照範囲	トリガー
channel	セット	NULL	チャンネル直下メッセージ（話題検出でフィルタ）	チャンネル画面から AI 質問ボタン (8-11)
thread	セット	セット	特定スレッド内の全会話	スレッド画面から AI 質問ボタン (8-12)
free	NULL	NULL	コンテキストなし。純粋な AI 会話	AI サイドバーから直接作成 (8-13)

CHECK 制約: thread_root_id がセットされている場合、channel_id も必ずセットされている。

---

3. API エンドポイント

3.1 エンドポイント一覧

メソッド	パス	説明	認証	レスポンス
POST	/api/ai/threads	スレッド作成	必要	JSON
GET	/api/ai/threads	スレッド一覧	必要	JSON
PATCH	/api/ai/threads/:id	タイトル更新	必要	JSON
DELETE	/api/ai/threads/:id	スレッド削除	必要	204
POST	/api/ai/ask	AI に質問	必要	**SSE ストリーミング**
GET	/api/ai/threads/:id/messages	会話履歴取得	必要	JSON
GET	/api/health	ヘルスチェック	不要	JSON
GET	/api/ready	レディネスプローブ	不要	JSON

3.2 リクエスト・レスポンス詳細

POST /api/ai/threads

```json
```

```
// Request
{
  "workspaceId": "ws-uuid",
  "title": "デプロイ手順について",
  "channelId": "ch-uuid",        // optional (channel/thread スコープ時)
  "threadRootId": "msg-uuid"     // optional (thread スコープ時)
}

// Response 201
{
  "id": "ai-thread-uuid",
  "workspaceId": "ws-uuid",
  "userId": "user-uuid",
  "title": "デプロイ手順について",
  "scope": {
    "type": "channel",
    "channelId": "ch-uuid"
  },
  "model": "gemini-2.5-flash",
  "createdAt": "2026-03-15T10:00:00Z",
  "updatedAt": "2026-03-15T10:00:00Z"
}
```

GET /api/ai/threads

```json
// Response 200
{
  "threads": [
    {
      "id": "ai-thread-uuid",
      "title": "デプロイ手順について",
      "scope": { "type": "channel", "channelId": "ch-uuid" },
      "model": "gemini-2.5-flash",
      "createdAt": "2026-03-15T10:00:00Z",
      "updatedAt": "2026-03-15T12:00:00Z"
    }
  ]
}
```

PATCH /api/ai/threads/:id

```json
```

```
// Request
{ "title": "新しいタイトル" }

// Response 200
{ "id": "...", "title": "新しいタイトル", ... }
```

DELETE /api/ai/threads/:id

```
// Response 204 No Content
```

POST /api/ai/ask

```json
// Request
{
  "threadId": "ai-thread-uuid",
  "message": "デプロイの手順を教えて"
}
```

```
// Response: SSE ストリーミング
Content-Type: text/event-stream

data: {"type": "text-delta", "content": "デプロイ"}
data: {"type": "text-delta", "content": "の手順"}
data: {"type": "text-delta", "content": "は以下の"}
data: {"type": "text-delta", "content": "通りです。"}
...
data: {"type": "done", "messageId": "ai-msg-uuid"}
```

GET /api/ai/threads/:id/messages

```json
```

```
// Response 200
{
  "messages": [
    {
      "id": "ai-msg-uuid-1",
      "role": "user",
      "content": "デプロイの手順を教えて",
      "createdAt": "2026-03-15T10:01:00Z"
    },
    {
      "id": "ai-msg-uuid-2",
      "role": "assistant",
      "content": "デプロイの手順は以下の通りです。\n\n1. ...",
      "metadata": {
        "inputTokens": 1250,
        "outputTokens": 340,
        "latencyMs": 820
      },
      "createdAt": "2026-03-15T10:01:02Z"
    }
  ]
}
```

---

4. AI 質問フロー（詳細設計）

4.1 処理ステップ

```
ユーザーが POST /api/ai/ask を呼ぶ
    │
    ▼
Step 1: スレッド取得 + スコープ判定
    │   threadStore.findById(threadId)
    │   → scope.type を確認 (channel / thread / free)
    │
    ▼
Step 2: コンテキスト取得 (free 以外)
    │   ┌─ Redis キャッシュ読取 (高速: < 5ms)
    │   │  chat:ctx:channel:{channelId} or chat:ctx:thread:{threadRootId}
    │   │  Sorted Set から ZRANGE で全件取得
    │   │
    │   └─ キャッシュミス → Go Backend API 呼出 (~50ms)
    │      POST /chatapp.message.v1.MessageService/GetMessages
    │      Connect protocol (HTTP/1.1 + JSON)
    │
```

```
    ▼
Step 3: 話題検出（channel スコープのみ）
    │   ┌─ Redis キャッシュ確認
    │   │  topic:boundary:channel:{channelId}
    │   │  キャッシュヒット + メッセージ数が同じ → boundaryIndex を使用
    │   │
    │   └─ キャッシュミス → Gemini Flash で境界判定
    │      直近 30 件を渡してメッセージ一覧から話題の境界を検出
    │      結果を TTL 5 分でキャッシュ
    │
    │   検出後: messages.slice(boundaryIndex) でコンテキストをフィルタ
    │
    ▼
Step 4: ユーザーメッセージ保存
    │   messageStore.save({ threadId, role: "user", content })
    │
    ▼
Step 5: AI 会話履歴取得
    │   messageStore.findByThread(threadId)
    │   → 過去の user/assistant メッセージ一覧
    │
    ▼
Step 6: プロンプト組立 + LLM 呼出（ストリーミング）
    │
    │   messages = [
    │     { role: "system", content: systemPrompt + chatContext },
    │     ...history,
    │     { role: "user", content: userQuestion }
    │   ]
    │
    │   llm.stream(messages)
    │   → ReadableStream を SSE として返す
    │
    ▼
Step 7: ストリーム完了後にアシスタントメッセージ保存
    │   stream を tee() して片方で全文を蓄積
    │   完了時に messageStore.save({ role: "assistant", content, metadata })
    │
    ▼
  クライアントが SSE を受信して逐次表示
```

4.2 プロンプト構造

```
[System]
あなたはチャットアプリに統合されたAIアシスタントです。
ユーザーの質問に的確に回答してください。
```

```
以下はユーザーが現在参加しているチャットの会話ログです。
質問に関連する情報があればこの文脈を踏まえて回答してください。

--- チャット会話ログ ---
田中: デプロイの手順ってどこに書いてある？
佐藤: wiki の /deploy/guide に書いてあるよ
田中: ありがとう、見てみる
有馬: terraform apply の前に plan は必ず実行してね
--- ログここまで ---

[User] デプロイの手順を教えて
[Assistant] (1つ前の会話)
[User] (前回の質問)
[Assistant] (前回の回答)
[User] デプロイの手順を教えて    ← 今回の質問
```

4.3 話題検出のプロンプト

```
[System]
あなたはチャットログの話題分析器です。
与えられたメッセージ一覧を分析し、最後の話題が始まったメッセージの番号を特定してください。

ルール:
- 話題の切り替わりとは、会話の主題が明確に変わった地点です
- 雑談の延長や関連する補足は「同じ話題」として扱ってください
- 最初から最後まで同じ話題の場合は 0 を返してください
- 数字のみ返してください（他の文字は不要）

[User]
[0] 田中: 今日のランチどこ行く？
[1] 佐藤: カレーがいいな
[2] 田中: いいね、あの店にしよう
[3] 有馬: デプロイの件なんだけど
[4] 有馬: staging で問題出てる
[5] 佐藤: どのサービス？
[6] 有馬: API サーバーの方

[Assistant]
3
```

この場合 boundaryIndex = 3 が返り、messages[3:] がコンテキストとして使われる。

---

5. データモデル

5.1 ai_threads テーブル

カラム	型	説明
id	UUID (PK)	
workspace_id	UUID (FK → workspaces)	
user_id	UUID (FK → users)	スレッドオーナー
channel_id	UUID (FK → channels, NULLABLE)	チャンネルスコープ時
thread_root_id	UUID (FK → messages, NULLABLE)	スレッドスコープ時
title	VARCHAR(200)	
model	VARCHAR(50)	デフォルト: "gemini-2.5-flash"
system_prompt	JSONB	カスタム指示（NULLABLE）
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

5.2 ai_messages テーブル

カラム	型	説明
id	UUID (PK)	
ai_thread_id	UUID (FK → ai_threads)	
role	ENUM('user', 'assistant')	
content	TEXT	
metadata	JSONB	トークン数, レイテンシ, コンテキスト範囲
created_at	TIMESTAMP	

5.3 metadata JSONB 構造

```json
```

```
{
  "inputTokens": 1250,
  "outputTokens": 340,
  "latencyMs": 820,
  "modelVersion": "gemini-2.5-flash",
  "contextRange": {
    "fromMessageId": "msg-uuid-start",
    "toMessageId": "msg-uuid-end",
    "messageCount": 8,
    "topicDetected": true
  }
}
```

---

6. Go Backend との連携

6.1 Redis コンテキストキャッシュ（Go が書込、AI Agent が読取）

キー	型	TTL	書込	読取
`chat:ctx:channel:{channelId}`	Sorted Set	24h	Go Backend	AI Agent
`chat:ctx:thread:{threadRootId}`	Sorted Set	24h	Go Backend	AI Agent
`topic:boundary:channel:{channelId}`	String (JSON)	5min	AI Agent	AI Agent

Sorted Set の構造:

- score: created_at の UnixTime ミリ秒
- member: JSON 文字列

```json
{
  "id": "msg-uuid",
  "userId": "user-uuid",
  "displayName": "有馬",
  "content": "デプロイの手順を確認したいです",
  "createdAt": "2026-03-15T10:30:00Z"
}
```

- Go Backend がメッセージ送信時に ZADD で追加
- 最大 30 件保持（ZREMRANGEBYRANK で古いエントリを削除）

- 新メッセージ時に `topic:boundary:channel:{channelId}` を DEL（話題境界キャッシュ無効化）

6.2 Go Backend API 呼出（キャッシュミス時のフォールバック）

```
POST http://go-backend:8080/chatapp.message.v1.MessageService/GetMessages
Content-Type: application/json

{
  "channelId": "ch-uuid",
  "pageSize": 30
}
```

Connect protocol（HTTP/1.1 + JSON）で呼ぶ。gRPC ではなく HTTP で直接呼べる。

6.3 JWT 共有

Go Backend と AI Agent は同じ JWT_SECRET を使う。 Go が発行したトークンを AI Agent が検証する。 トークン payload: `{ sub: "<userID>", exp: <timestamp> }`

---

7. 認証

全 /api/ai/* エンドポイント（health, ready を除く）で JWT 検証が必要。

- Authorization ヘッダーから `Bearer <token>` を取得
- jose ライブラリで検証（Go Backend と同じ秘密鍵）
- payload の `sub` からユーザー ID を取得
- Hono の context にユーザー情報をセット

---

8. ディレクトリ構成

```
agent/
├── src/
│   ├── domain/
│   │   ├── entities/
│   │   │   ├── ai-thread.ts
│   │   │   ├── ai-message.ts
│   │   │   ├── chat-context.ts
│   │   │   └── scope.ts
│   │   ├── types/
│   │   │   └── index.ts
│   │   └── ports/
```

```
│   │       ├── thread-store.port.ts
│   │       ├── message-store.port.ts
│   │       ├── chat-context.port.ts
│   │       ├── context-cache.port.ts
│   │       ├── llm-gateway.port.ts
│   │       └── topic-detector.port.ts
│   │
│   ├── application/
│   │   └── usecases/
│   │       ├── ask.usecase.ts
│   │       ├── manage-thread.usecase.ts
│   │       └── get-history.usecase.ts
│   │
│   ├── infrastructure/
│   │   ├── llm/
│   │   │   ├── gemini-api.gateway.ts
│   │   │   ├── vertex-ai.gateway.ts
│   │   │   └── llm-gateway.factory.ts
│   │   ├── mastra/
│   │   │   ├── index.ts
│   │   │   ├── agents/copilot.agent.ts
│   │   │   ├── tools/
│   │   │   │   ├── fetch-context.tool.ts
│   │   │   │   └── detect-topic.tool.ts
│   │   │   └── adapters/
│   │   │       ├── llm-gateway.adapter.ts
│   │   │       └── topic-detector.adapter.ts
│   │   ├── persistence/
│   │   │   ├── redis/
│   │   │   │   ├── client.ts
│   │   │   │   └── context-cache.impl.ts
│   │   │   └── mastra-memory/
│   │   │       ├── thread-store.impl.ts
│   │   │       └── message-store.impl.ts
│   │   └── go-client/
│   │       └── chat-context.impl.ts
│   │
│   ├── interface/
│   │   └── http/
│   │       ├── app.ts
│   │       ├── routes/
│   │       │   ├── threads.route.ts
│   │       │   ├── chat.route.ts
│   │       │   ├── health.route.ts
│   │       │   └── readiness.route.ts
│   │       └── middleware/
│   │           ├── jwt-auth.ts
```

```
│   │           ├── error-handler.ts
│   │           └── request-logger.ts
│   │
│   ├── di/container.ts
│   ├── shared/
│   │   ├── errors.ts
│   │   ├── logger.ts
│   │   └── config.ts
│   └── index.ts
│
├── package.json
├── tsconfig.json
├── biome.json
├── Dockerfile
└── .env.example
```

---

9. 環境変数

```bash
# サーバー
PORT=3001

# データベース（Mastra Memory 用）
DATABASE_URL=postgres://user:pass@host:5432/chatapp?sslmode=disable

# Redis（Go が書き込んだコンテキストキャッシュの読取）
REDIS_URL=redis://localhost:6379

# 認証（Go Backend と同じ秘密鍵）
JWT_SECRET=same-as-go-backend

# Go Backend（コンテキスト取得のフォールバック）
GO_BACKEND_URL=http://localhost:8080

# Gemini API（ローカル開発）
GOOGLE_GENERATIVE_AI_API_KEY=your-api-key

# Vertex AI（本番 — Workload Identity で自動認証、キー不要）
# GCP_PROJECT_ID=your-project
# GCP_LOCATION=asia-northeast1
```

---

10. 非機能要件

10.1 パフォーマンス

指標	目標
コンテキスト取得（Redis ヒット）	< 5ms
コンテキスト取得（Go API フォールバック）	< 100ms
話題検出（Gemini Flash）	< 1s
SSE 初回トークン到着（Time to First Token）	< 2s
スレッド CRUD	< 100ms

10.2 制限

項目	値
コンテキストの最大メッセージ数	30 件
コンテキストキャッシュ TTL	24 時間
話題境界キャッシュ TTL	5 分
AI 会話のストリーミングタイムアウト	60 秒
スレッドタイトル最大長	200 文字
メッセージ最大長	10,000 文字

10.3 エラーハンドリング

エラー	HTTP ステータス	対応
JWT 未送信 / 無効	401 Unauthorized	
スレッドが見つからない	404 Not Found	
他ユーザーのスレッド操作	403 Forbidden	
LLM API エラー	502 Bad Gateway	リトライ 1 回、それでもダメなら返す
Redis 接続エラー	Go API にフォールバック	キャッシュなしで動作継続
Go Backend 接続エラー	503 Service Unavailable	コンテキストなしでは回答しない

エラー	HTTP ステータス	対応
バリデーションエラー	400 Bad Request	Zod のエラーメッセージを返す

---

11. import 制約

```
domain/**          → 外部ライブラリ禁止。TypeScript 標準のみ。
application/**     → domain/ + shared/ のみ。
infrastructure/** → domain/ + shared/ のみ。
interface/**       → application/ + shared/ のみ。
di/container.ts    → 全レイヤー（組立のみ）。
shared/**          → 外部ライブラリ + 標準のみ。
```

---

12. 実装順序（推奨）

```
Step 1: セットアップ（ディレクトリ生成、依存インストール、tsconfig）
Step 2: shared/（config, logger, errors）
Step 3: domain/（entities → types → ports）
Step 4: application/（usecases — ports を参照するだけなので先に書ける）
Step 5: infrastructure/persistence/（Redis client, context-cache, mastra-memory）
Step 6: infrastructure/llm/（gemini-api, vertex-ai, factory）
Step 7: infrastructure/mastra/（index, copilot.agent, tools, adapters）
Step 8: infrastructure/go-client/
Step 9: interface/（middleware → routes → app.ts）
Step 10: di/container.ts（全部を繋ぐ）
Step 11: index.ts + ローカル起動テスト
```